

NTNU

Det skapende universitet

TDT4110 ITGK:

Funksjoner, virkemåte og bruk

Læringsmål

- Funksjoner
  - Lære hvordan man kan dele opp programmer i funksjoner
  - Lære å definere og kalle funksjoner
  - Lære om overføre argumenter til funksjoner
  - Beherske returverdier og «side-effekter»

  Navnerom og skop
  - Lære om lokale og globale variabler
  - LEGB regelen: Local – Enclosing – Global – Built-in

  Moduler
  - Hva er moduler og hvordan bruke disse

Introduksjon til funksjoner

- En **funksjon** er en gruppe av programlinjer som eksisterer i et program med den hensikt å gjøre en spesifikk oppgave.
- De fleste programmer er så store at det er fornuftig å del de opp i mindre deler (**modularisering)**
- Funksjoner hjelper oss å
  - ***organisere koden*** i mer håndterbare deler
  - ***abstrahere,*** skille mellom *hva* som skal gjøre og *hvordan* det skal gjøres («gjemme» implementasjonsdetaljer).

Forelesninger\uke38\oppdeling_i_funksjoner_eksempel.ipynb

Konsept – dele opp koden

Programmet uten funksjoner:
Mindre oversiktlig

```
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
```

Programmet delt opp i funksjoner:
Mer oversiktlig

```
def function1(x):
  statement
  statement
  statement
```

```
def function2():
  statement
  statement
  statement
```

```
def main():
  statement
  function1(3)
  statement
  function1(5)
  function2()
  statement
```

Fordeler med funksjoner:

- Mer oversiktlig kode
- Gjenbruk: Slipper å skrive samme kode flere ganger
- Bedre testing: Testing og debugging er enklere med funksjoner (kan teste én og én funksjon)
- Raskere utvikling: Kode som trengs i flere deler av systemet kan brukes flere ganger
- Bedre støtte for samarbeid: Kan fordele funksjoner på flere programmerere
- Abstraksjon: En funksjon kan implementere en abstraksjon. Gode abstraksjoner gjør det lettere for oss å tenkte og forstå koden!

Top-down design

- Top-down design er en måte å tenke på når man skal designe programmer:
  - Hovedoppgaven til programmet blir brutt ned i en serie av deloppgaver
  - Hver deloppgave blir analysert for å se om de kan brytes ned eller ikke
  - Når alle deloppgaver er identifisert, blir koden skrevet
- I top-down design bygger man skjelettet først og deretter fyller inn detaljene

Steg/prinsipper for oppdeling i funksjoner

1. Fjerne duplisert kode
   - Identifiser duplisert eller nesten like kode-segmenter.
   - Hva er likt, hva varierer?
   - Lag en funksjon for den dupliserte koden
2. En funksjon skal gjøre én ting, og gjøre det bra!
   - Hver funksjon skal ha «én oppgave»
   - Har den flere oppgaver, splitt
   - Løs oppgaven ved å sette sammen funksjonskall
3. Gjør funksjoner mer gjenbrukbare ved å gjøre dem mer generelle

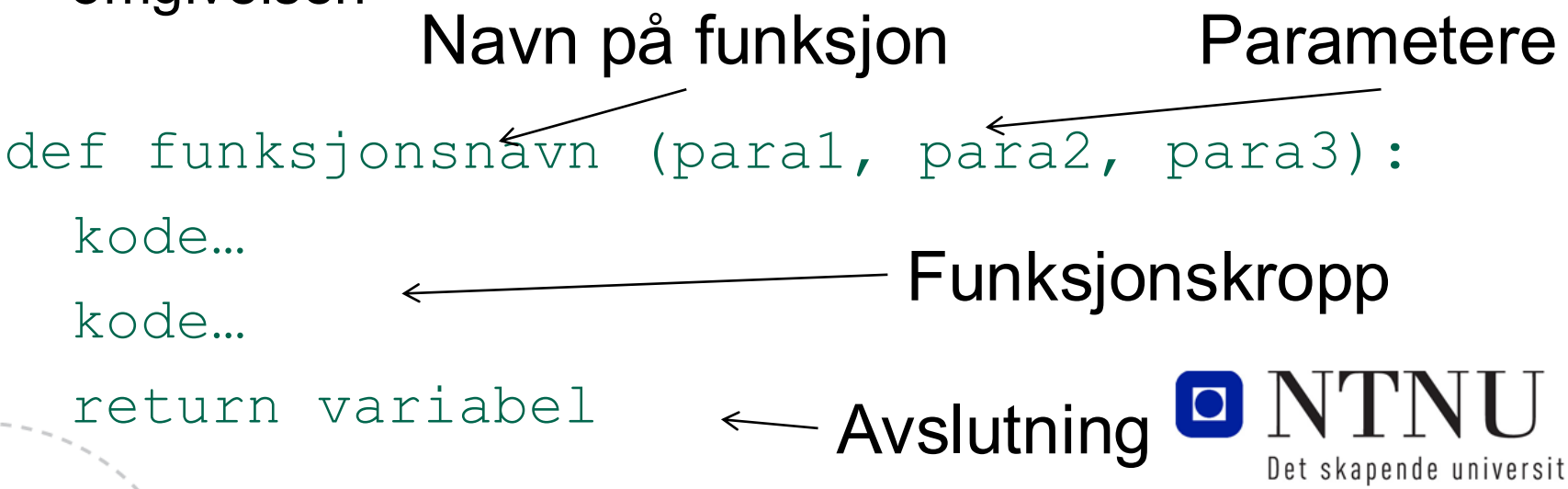
Se Forelesninger\uke38\dekomponering_i_funksjoner.ipynb

Funksjonsnavn

- Reglene for funksjonsnavn er de samme som for variabelnavn.
- En «kaller» (utfører) en funksjon ved skrive funksjonsnavnet etterfulgt av paranteser ( ).
- Et eksempel på en innebygd funksjon er print:
  - `print("Hello, World", end = "! ")`
- Det som står i parantes i kallet, er *argumenter* til funksjonen. Dette er input til funksjonen, som *tilordnes* input-parametrene til funksjonen.

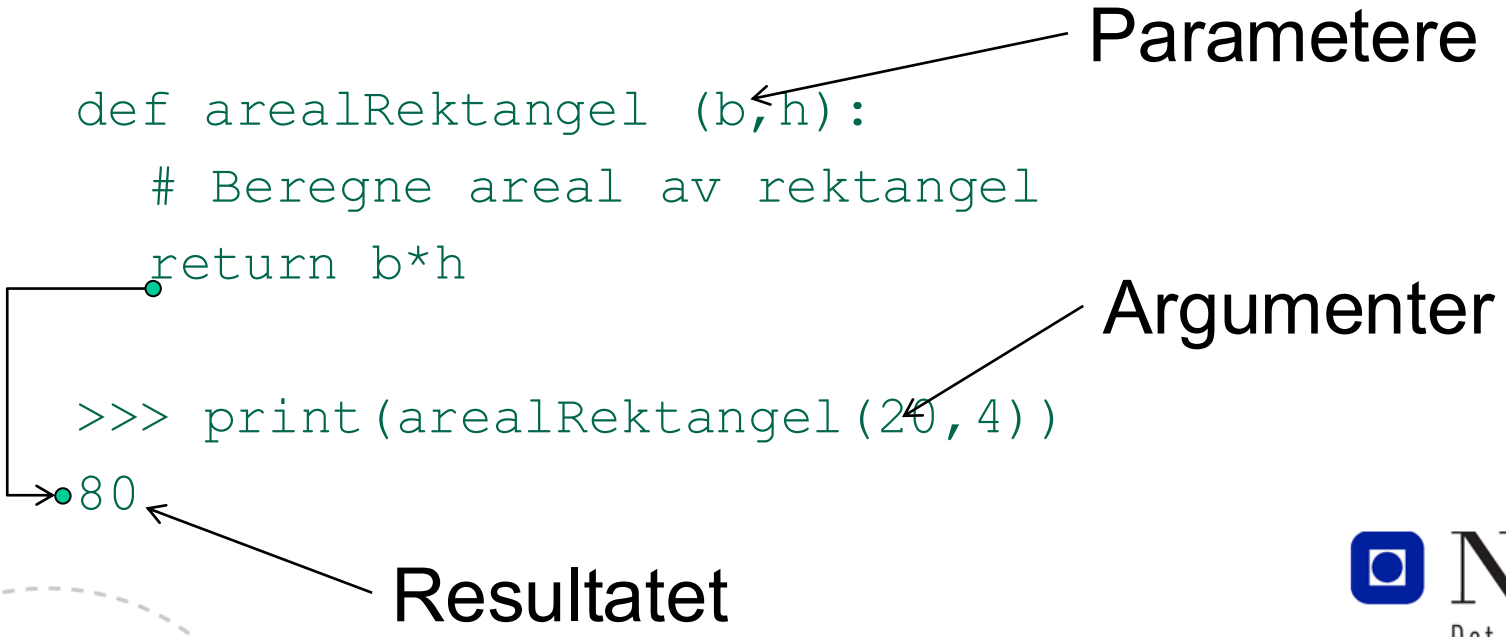
Oppbygning av en funksjon.

- Når man definerer/lager en funksjon bruker man det reserverte ordet `def`
- All koden i funksjonen (kroppen) skrives i *innrykk*.
- Avslutter med `return` (når noe skal returneres)
- Funksjoner kan også skrives i den interaktive omgivelsen



Kall av funksjoner

- Ved kall av en funksjon brukes *funksjonsnavnet*, samt en liste av *argumenter* (verdier) som matcher navnet på funksjonen og lista av parametere:



Funksjoner med eller uten returverdi

- print() gir oss ikke noen verdi tilbake. Den skriver kun noe til konsollet.
- Funksjoner kan også ha et ***resultat***
- Et eksempel er input-funksjonen

```
svar = input("Hva heter du? ")
```

- Den både skriver noe til konsollet («ledeteksten»), og har som resultat det som brukeren skriver.
- Dette kaller vi ***returverdien*** til funksjonen.
- Returverdien *blir verdien til* funksjons-kallet, akkurat som i matematikk
  - Etter funksjonskallet over, blir verdien tilordnet variabelen **navn.**

Returverdier

- En funksjon avsluttes når programpekeren kommer til en **return-setning, eller kommer til siste linje i funksjonskroppen.**
- Etter return kan det stå et uttrykk (verdi, variabelnavn) som blir det som blir sendt tilbake til funksjonskallet (output av funksjonen)

  return <uttrykk>
- Hvis det ikke er oppgitt noe uttrykk, eller hvis programkjøringen kommer til slutten av funksjonen, så returneres None

  (som representerer «ingenting»)

•Når programkjøringen kommer til «return <uttrykk>», beregnes uttrykket, verdien returneres som verdien av funksjonen, og programkjøringen fortsetter der hvor funksjonen ble ***kalt.***

Argument og parameter

En funksjon kan sees som nesten som et miniprogram i programmet, som har sine input og output.

Input til funksjonen kan være som

- input parameter spesifisert i definisjonen til funksjon
- en global variable definert utenfor funksjonen
- Annen input fra terminal eller fil

Output kan være

- Returverdi
- Lagre noe I en global variable
- Skrive ut til terminal eller fil

Input-parameter og argumenter

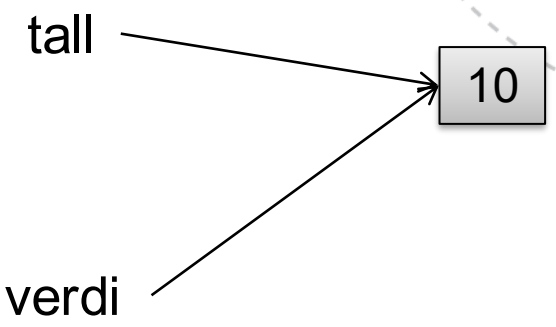
Den tydeligste måten å ha input til funksjoner er å angi dem som input-***parametere***.

- En **parameter** er en variabel som mottar input fra funksjons-kallet.
- Vi kaller den verdien som mottas for **argumentet.**
  - Parameteren brukes som en vanlig variabel, men kun inne i funksjonen, dvs. skopet (synligheten) til parameteren er inne i funksjonen.
  - Man kan gjøre endringer i verdien til en parameter inne i en funksjon, men for ikke-muterbare datatyper (int, float, str, bool) har dette ikke noen effekt utenfor funksjonen.

Overføring av verdier til funksjoner

```
def vis_dobbelte(tall):
    resultat = tall * 2
    print(resultat)

verdi = 10
vis_dobbelte(verdi)
```

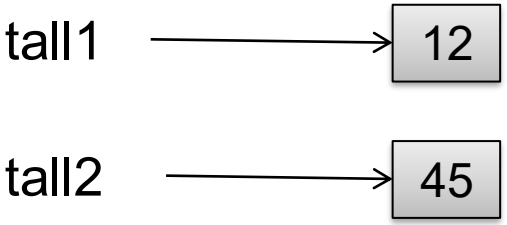


Sende over flere argumenter

- Funksjoner kan ha fra ingen til mange parametere.
- Vanlig måte å sende over flere argumenter til en funksjon er å sende inn verdier i riktig rekkefølge:

```
def vis_sum(tall1, tall2):
    resultat = tall1 + tall2
    print(resultat)

vis_sum(12,45)
```

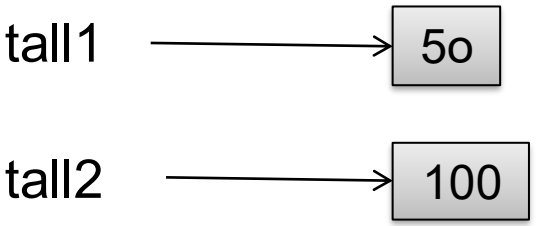


Bruke nøkkelord for argumenter

- Kan også spesifisere parameternavn som argumenter:

```
def skriv_tall(tall1, tall2):
    print('Tall 1:',tall1)
    print('Tall 2:,'tall2)

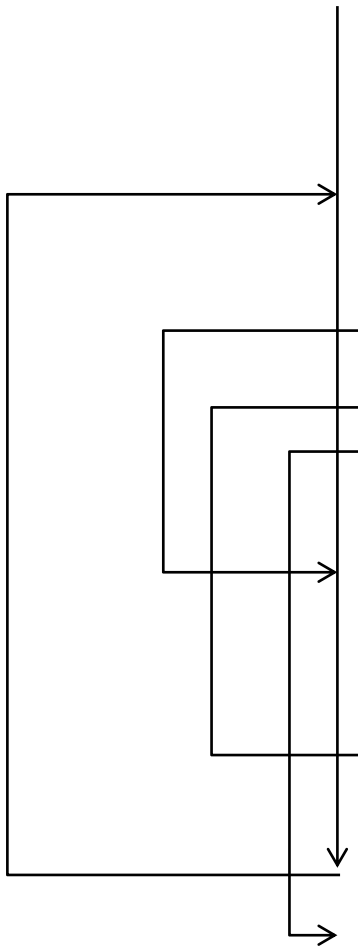
skriv_tall(tall2=100,tall1=50)
```



Bruk av flere funksjoner

- Programmer kan med fordel bygges opp ved å dele det opp i funksjoner som til sammen utfører en oppgave.
- En god praksis er å ha en hovedfunksjon som kalles main:
  - main-funksjonen inneholder hovedlogikken i programmet og gjengir overordnet struktur i programmet
  - main-funksjonen kaller andre funksjoner som utfører hver sin deloppgave.
  - Deloppgavene kan igjen bestå av kall til andre funksjoner, osv.

Kall av flere funksjoner



```
# Programmet har to funksjoner

# Definisjon av funksjonen main
def main():
        print('Melding til deg:')
        melding()
        print('Ha det!')

# Definisjon av funksjonen melding
def melding():
        print('Dum og deilig')
        print('Juba, juba!')

# Kall av funksjonen
main()
```

Lokale variabler

- En lokal variabel blir opprettet inni en funksjon og kan ikke aksesseres (fås tak i) av kode utenfor funksjonen.
- Begrepet lokal variabel indikerer at variabelen kan kun brukes lokalt i funksjonen

```
def hent_navn():
    navn = input('Skriv navnet: ') # Lokal variabel!
    # Variabelen navn kan kun brukes her!!!

hent_navn()
print('Hallo',navn)     # gir feilmelding!!!
```

Oppgave: Funksjoner med parametere



- Skriv koden til funksjonen areal_sirkel:
  - Tar parameterene r (radius) og pi
  - Beregner arealet av en sirkel ved:

$$A = \pi \bullet r^2$$

  - Returnere arealet

NB! Forskjell på å printe verdien og å returnere den.

Flere return-setninger i en funksjon

- Det kan være flere return-setninger.
- Man hopper ut av funksjonen når det påtreffes en return-setning, uansett hvor i koden man er.

```python
# Regner ut karakterer ut fra NTNUs prosentpoengmetode
# Input : poeng
# Output : karakter

def karakterBeregning(poeng):
    if poeng >= 89:
        karakter = 'A'
    elif poeng >= 77:
        karakter = 'B'
    elif poeng >= 65:
        karakter = 'C'
    elif poeng >= 53:
        karakter = 'D'
    elif poeng >= 41:
        karakter = 'E'
    else:
        karakter = 'F'
    return karakter

poeng = int(input('Oppgi poengsum: '))
print(f'Karakteren ble {karakterBeregning(poeng)}')
```

```
>>> %Run 24-1.py
Oppgi poengsum: 47
Karakteren ble E
>>> %Run 24-1.py
Oppgi poengsum: 78
Karakteren ble B
>>>
```

```python
# Regner ut karakterer ut fra NTNUs prosentpoengmetode
# Input : poeng
# Output : karakter

def karakterBeregning(poeng):
    if poeng >= 89:
        return 'A'
    elif poeng >= 77:
        return 'B'
    elif poeng >= 65:
        return 'C'
    elif poeng >= 53:
        return 'D'
    elif poeng >= 41:
        return 'E'
    else:
        return 'F'

poeng = int(input('Oppgi poengsum: '))
print(f'Karakteren ble {karakterBeregning(poeng)}')
```

```
>>> %Run 24-1.py
Oppgi poengsum: 78
Karakteren ble B
>>> %Run 24-2.py
Oppgi poengsum: 98
Karakteren ble A
>>>
```

Å returnere *boolske verdier*

- Boosk funksjon: returnerer enten `True` eller `False`
  - Brukes til å teste et kriterium for beslutningsstrukterer og repetitive strukturerer
    - F.eks å repetere handlingen i en løkke til en teller har nådd et visst tall
    - Vanlige beregninger, som å sjekke om et tall er partall kan enkelt repeteres gjennom å kalle på en funksjon
  - Brukes til å forenkle sjekk av inndata for gyldighet
    - Du kan spørre om verdier, og bare godta enkelte verdier ved å returnere `True` hvis inndata er en av disse
    - Vi ser på et eksempel!

Oppgave: funksjon med boolsk retur-verdi



- Lag funksjonen «er_rettvinklet» som sjekker om de tre lengdene a,b og c kan utgjøre sidene i en rettvinklet trekant.
- Returner True dersom de utgjør det, False ellers.
- Benytt deg av `def` og `return`
- Skriv også koden for følgende:
  - Spør brukeren om sidene
  - Skriv ut
  - Bruk `input` og `print`

Å returnere flere verdier

- I Python kan en funksjon returnere flere verdier
  - Spesifisert etter return-uttrykket, atskilt av kommaer
    - Format: `return uttrykk1, uttrykk2`, etc.
  - Når du kaller på en slik funksjon i et tilordningsuttrykk trenger du en egen variabel på venstresiden av = operatoren for å ta i mot hver returnert verdi:

```
def get_name():
    first_name = input("First name: ")
    last_name = input("Last name: ")
    return first_name, last_name

first, last = get_name()
print(f"Given name: {first} Family name: {last}")
```

  - (Verdier separert med komma definerer her et «tuppel», som vi skal komme tilbake til senere.)

Globale variabler og globale konstanter

- En global variabel kan aksesseres av alle funksjoner i en programfil (pyton-fila der koden er lagret)
- En global variabel må opprettes utenom en funksjon

```
# Opprett global variabel
verdi = 10

def vis_verdi():
    print(verdi)  # Fungerer fordi verdi er global

vis_verdi()
```

NB! Globale variabler og Jupyter notebook

- Alle variable som defineres rett i en celle er globale for hele notebook'en, etter at cellen er utført.
- De er fortsatt definert selv om cellen slettes, eller vi endrer navn på variablen i cellen senere.
- Hvis en har jobbet lenge med en notebook, kan det være en god ide å restarte Python-tolkeren (restart kernel), og kjøre alle kode-cellene på nytt.

Endre på global variabel i funksjon

- Global variabel kan endres i funksjon ved å bruke kodeordet **`global`** inne i funksjonen

```
tall = 5 # Opprett global variabel

def main(): # Kan endre variabel tall her!
    global tall # Kan nå endre på variabelen tall
    tall= eval(input('Skriv et tall: '))
    vis_tall()

def vis_tall(): # Kan ikke endre variabel tall her!
    print('Tallet er:',tall)

main()
```

Bruk av globale variabler og konstanter

- I store programmer gjør bruk av globale variabler det vanskelig å finne feil (debugge) og bør derfor unngås.
- Man bør spesielt være forsiktig med å endre på globale variabler inne i funksjoner, som gjør koden svært rotete.
- Bruksområde for globale variabler er å definere konstanter som ikke skal endres i koden:
  - Konstanter er variabler som ikke endrer verdi
  - Konstanter skrives gjerne kun med store bokstaver:

```
MOMS = 0.25 # Endres sjeldent
MAT_MOMS = 0.125 # Endres sjeldent
```

Navnerom (namespace)

I Python er det flere typer navn: variabel-navn, funksjonsnavn (og klassenavn, osv. som ikke er pensum).

- Et navnerom er en kobling mellom et sett med navn og dataobjekter (som de er tilordnet).
- Dvs. hva er det et navn faktisk referer til et gitt sted i koden?
- Ikke alle navn er kan brukes alle steder i koden, som vi har sett med lokale variabelnavn

Skoping (synlighet)

- Skopet til et navnerom betegner de delene av programmet hvor navnene i navnerommet er direkte tilgjengelige.
- Et navn (variabelnavn, funksjonsnavn etc) er synlig bare i kodelinjene innenfor skopet.
- En variabel er heller ikke synlig før variabelen er opprettet:

```
def feil_funksjon():
    print('Verdien er: ',verdi) # gir feilmelding
    verdi = 99
```

Skoping (synlighet)

- **Innebygd skop**: Alltid tilgjengelig. Inneholder bla. Standard funksjoner som ikke må imprteres (print() etc.)
- **Globalt skop**: Også kalt modul-skop. Navn definert utenfor funksjoner
- **Lokalt skop**: Navn definert inne i funksjoner
- **Enclosing scope**: Et lokalt skop som omslutter et annet lokalt skop, hvis f.eks. en funksjon inneholder en annen funksjonsdefinisjon.

Skoping forts

- Skop handler om hvor i koden et ***navn*** er tilgjengelig.
- Et dataobjekt kan aksesseres hvis det er et navn som referer til objektet i det gitte skopet.
- Ved passende tilordning så kan et navn referere til data/objekter opprettet hvor som helst i koden fra hvor som helst.

Lokale skop er adskilt

To separate funksjoner har forskjellig lokale skop

```
# Definisjon av funksjonen nordnorge
def nordnorge():
        fugl = 120    # Variabelen fugl kun for nordnorge()
        print('Nord-Norge har',fugl,'fugletyper')
# Definisjon av funksjonen vestlandet
def vestlandet():
        fugl = 147    # Variabelen fugl kun for vestlandet()
        print('Vest-landet har',fugl,'fugletyper')

nordnorge()
vestlandet()
```

Variablene fugl i de to funksjonene er to forskjellige variabler og opphører etter funksjonen er ferdig utført sin kode.

Lokale skop er adskilt forts

Også hvis vi kaller en funksjon fra en funksjon, så har ikke den kalte funksjonen noen tilgang til det lokale skopet til kaller-funksjonen:

```
# Definisjon av funksjonen nordnorge
def nordnorge():
        fugl = 120    # Variabelen fugl kun for nordnorge()
        print('Nord-Norge har',fugl,'fugletyper')
# Definisjon av funksjonen vestlandet
def vestlandet():
        fugl = 147    # Variabelen fugl kun for vestlandet()
        print('Vest-landet har',fugl,'fugletyper')

nordnorge()
vestlandet()
```

Variablene fugl i de to funksjonene er to forskjellige variabler og opphører etter funksjonen er ferdig utført sin kode.

Nonlocal

- Ligner på global, men brukes for å skrive til lokal variabel i omsluttende lokalt skop (enclosing scope)

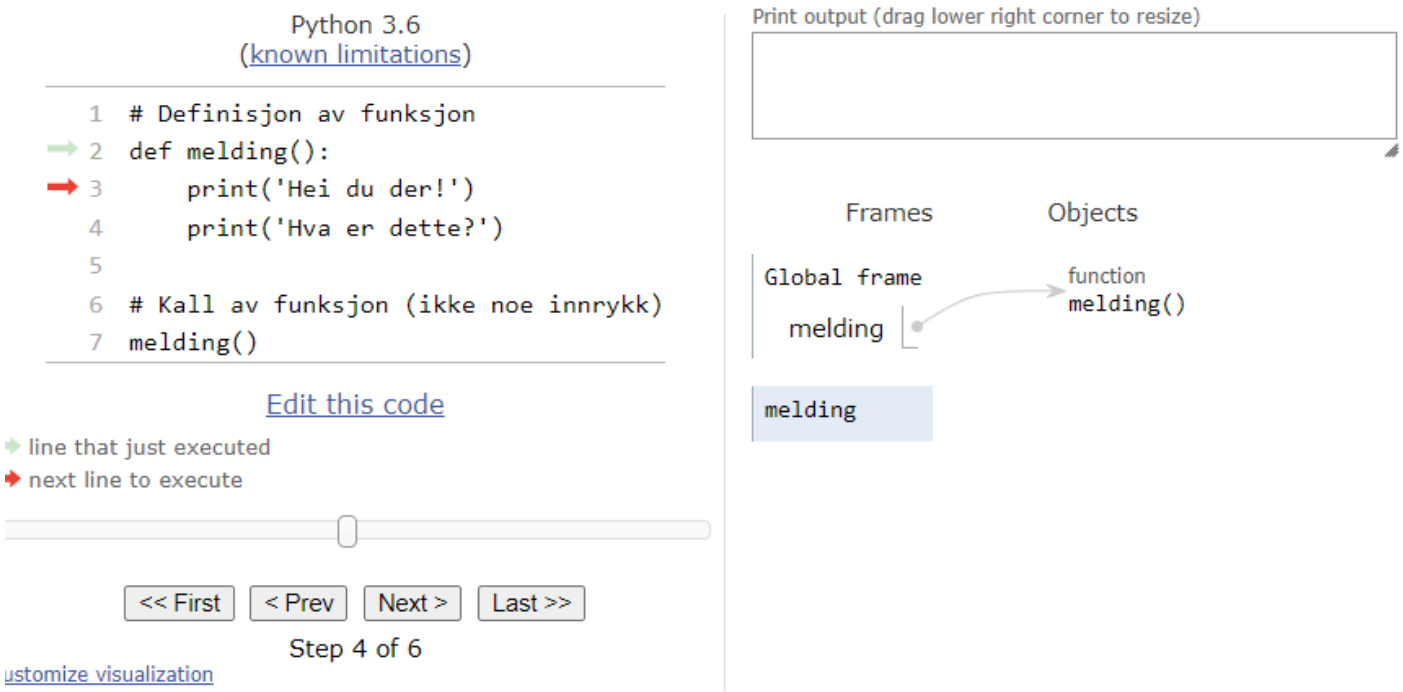
```
def ytre_funksjon():
    x = 1
    def indre_funksjon():
        nonlocal x
        x += 1
    indre_funksjon()
    print(x)

ytre_funksjon()
```

Visualisere Python-kjøring

Det er lettere å bruke en IDE (Thonny e.l.) for å se trinn for trinn hva som skjer i et program. Et annet verktøy er

https://pythontutor.com/



NTNU
Det skapende universitet

Moduler og pakker (packages)

Sammen med funksjoner, så brukes moduler og pakker for å modularisere, dvs. organisere koden i separate mer håndterbare og gjenbrukbare deler.

En modul er ikke annet enn en .py-fil med Python kode.

En pakke (package) er en samling moduler, organisert i mapper.

- Standardbibliotek: Moduler med funksjoner som kommer med Python-installasjonen
  - Noen er lastet inn ved oppstart (f.eks. print, input, …)
  - Mange ligger i moduler som må importeres først

Definere egne moduler

- Vi definerer en Python-fil som inneholder funksjoner og annen kode
- Python leter på gitte steder etter moduler, inkludert mappen/folderen hvor programmet kjører fra.
- Hvis modulen inneholder kommandoer i globalt skop, så utføres disse én gang ved import, dvs ikke på nytt hvis samme modul importeres flere ganger.

Eksempel på egendefinert modul

```python
import minmodul as mm

print(mm.favoritt_tall)

tall1 = int(input("Tall 1: "))
tall2 = int(input("Tall 2: "))

print(f"Største felles divisor er
{mm.gcd(tall1, tall2)}")
```

```python
# DETTE ER FILEN minmodul.py

print('Dette er "minmodul"')
favoritt_tall = 6.28

def gcd(a,b):
    a,b = abs(a), abs(b)
    a,b = min(a,b), max(a,b)
    while b > 0:
        a,b = b, a%b
    return a

if __name__ == "__main__":
    print(f"Favoritt-tallet mitt er
{favoritt_tall}")
```

I koden over til venstre, så vil import av minmodul.py:

- Printe ut: Dette er «minmodul»
- Definere variabelen favoritt_tall med verdien 6.28
- Definere funksjonen gcd
- IKKE utføre koden som starter med if __name__
  - Denne utføres kun hvis vi kjører minmodul.py direkte

Kommentarer versus dokumentasjon av kode

- **Kommentarer** i koden er for å forklare (særlig hvorfor), for *de som skal lese / vedlikeholde / videreutvikle koden* Det er ofte deg selv! Du vil bli overrasket over hvor lite du husker etter en stund, særlig hvis du har vært kreativ….
- **Dokumentasjon** er for å forklare hvordan koden skal brukes. En måte å gjøre det på er å legge til **docstrings**,

Oppsummering

- Funksjoner er en måte å dele opp programmer i mindre deler som gir mange fordeler.
- En funksjon må defineres og består av hode og kodeblokk:

```
def funksjonsnavn():
    kode…
```

- En funksjon kalles (kjøres) med funksjonsnavnet:

```
funksjonsnavn()
```

- Funksjoner kan ha lokale variabler som kun kan brukes (skop/synlighet) inne i funksjonen.

Oppsummering – funksjonsinput

- Funksjoner kan ta imot verdier ved hjelp av parametere.
  - En parameter er en variabel som tar imot en verdi når den blir kalt:

```
funksjon(parameter1, parameter2): # Variabler
    kode…
```

- Verdier kan overføres til funksjoner ved hjelp av argumenter:

```
funksjon(argument1, argument2) # Verdier
```

- Funksjoner har tilgang til å lese globale variable
- Det beste er vanligvis å bruke overføring via parametre
  - Mer oversiktlig, vi vet hva funksjonen avhenger av

Oppsummering – verdier fra en funksjon

- Funksjoner returnerer verdier ved return-setninger
  - Trenger ikke return-seting, da returneres None
  - Kan ha mange return-setninger.
  - Kan returnere flere verdier som tupler
- En funksjon kan også endre globale variable
  - Da må variabelen deklareres med nøkkelordet global
- Input-argumenter som er ***muterbare***, kan endres inne i en funksjon, effekten er synlig utenfra.
- Å bruke return er vanligvis en foretrukket måte å få verdier fra en funksjon.
  - Mer tydelig hva en funksjon gjør

Oppsummering - moduler

- Å bruke bibliotek-funksjoner og import-uttrykket
  - Modulene random og math, numpy, etc
- Å gruppere dine egne funksjoner i moduler
  - Moduler er bare .py-filer med kode